

Experiential Learning Project
Assessment-I on
“Expression Tree Generator and
Evaluator ”

under the course
Python ProgrammingLanguage(N-PCCCM304P)

presented by
Pranjal Parihar
CM25101



Under the Guidance
of
Mrs. Mayuri Getme

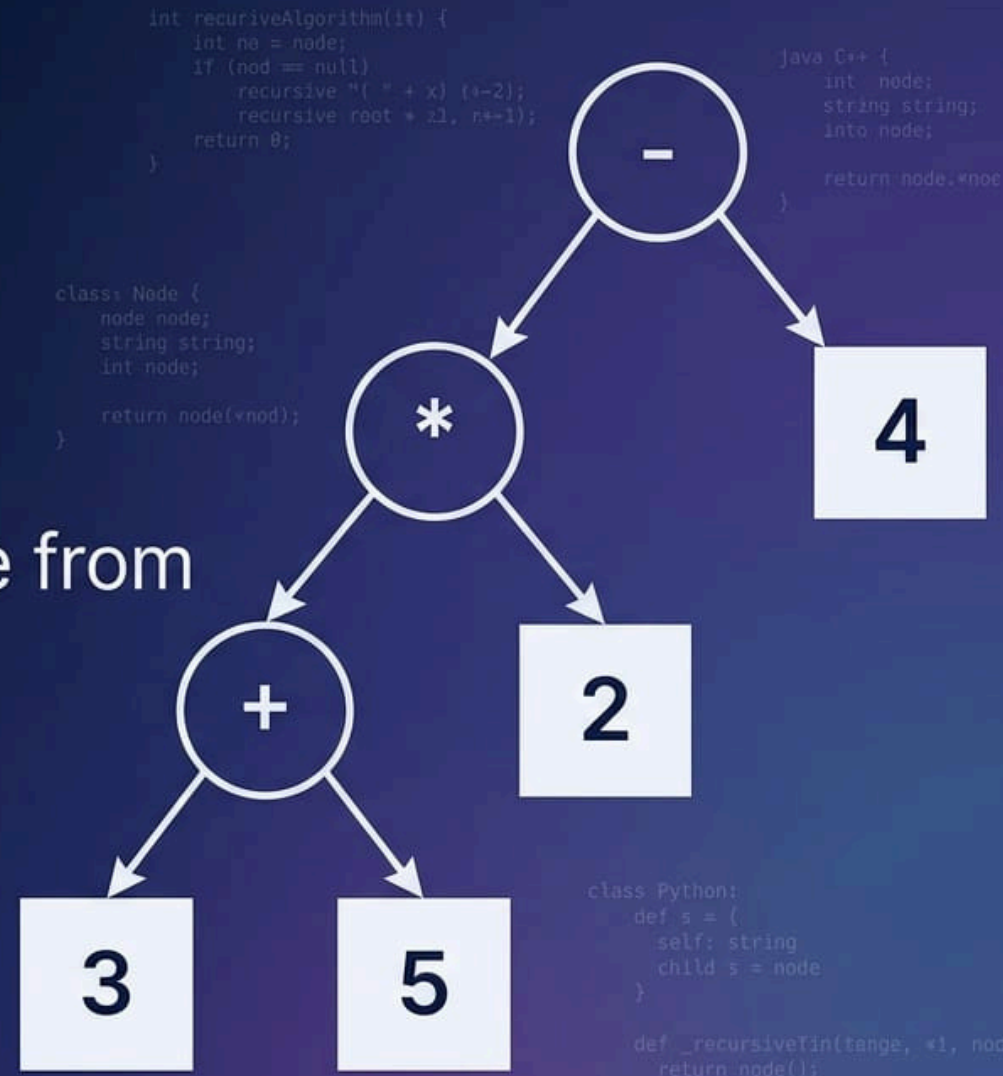
Session:2026-27 (ODD)
Department of CSE(AI&ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND
RESEARCH,NAGPUR

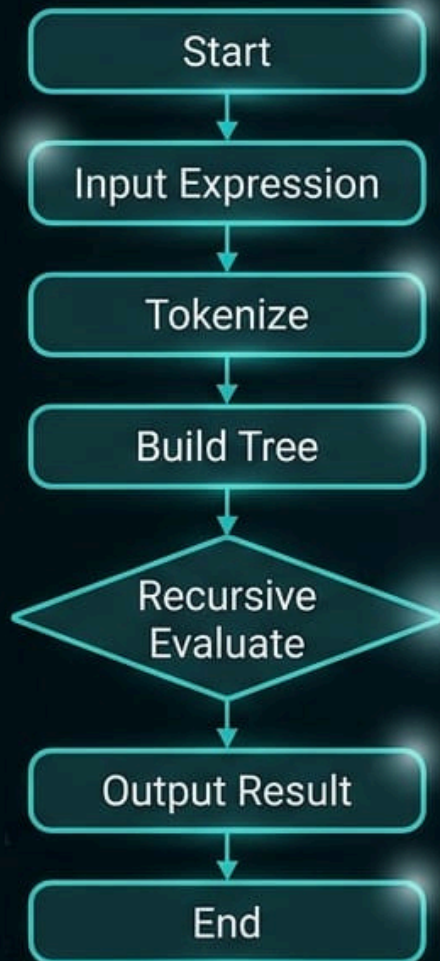
An Autonomous Institute, Affiliated to RTMNU, Nagpur

Problem Statement

Create an application that constructs an expression tree from arithmetic expression and evaluates the result using recursive tree traversal techniques.



Flowchart / Pseudocode



```
function evaluate(node):  
    if node is leaf: return node.value  
    left = evaluate(node.left)  
    right = evaluate(node.right)  
    return apply(node.operator, left, right)
```

Expression Tree Generator and Evaluator

Constructing Expression Trees from Arithmetic Expressions and Evaluating Results using Recursive Tree Traversal Techniques



GITHUB ACCOUNT, REPOSITORY CREATION & COLLABORATION



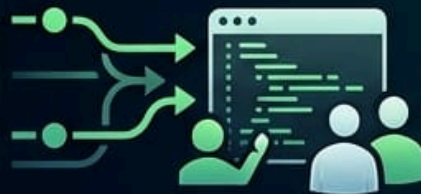
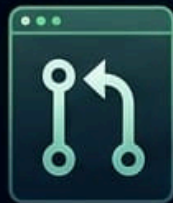
CREATE GITHUB ACCOUNT & NEW REPOSITORY

Sign up user profiles, initialize public or private project storage.



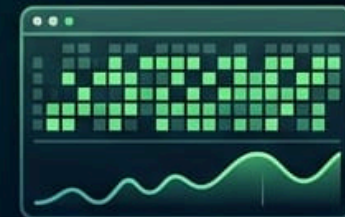
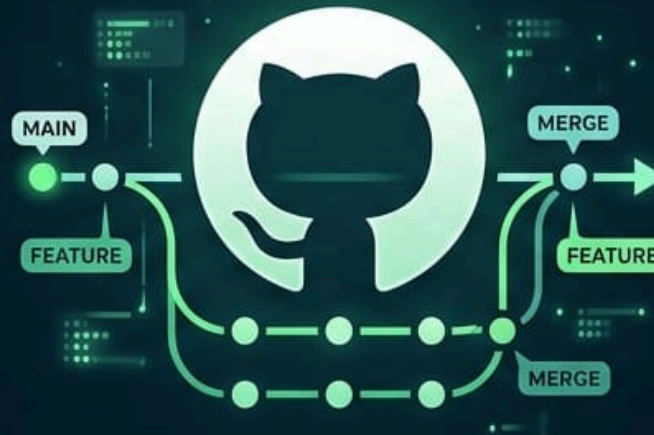
UPLOAD CODE WITH MEANINGFUL COMMITS

Push local code changes, provide descriptive commit messages.



3. COLLABORATION (PULL REQUESTS, BRANCHING)

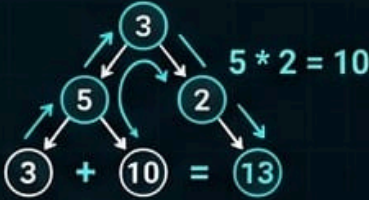
Work with feature branches, review contributions, merge code.



4. INDIVIDUAL CONTRIBUTION TRACKING

Monitor individual performance, visualize coding activity over time.

Algorithm

- 1 Tokenize the input arithmetic expression 
Input: "3 + 5 * 2" → [3, +, 5, *, 2]  
- 2 Convert infix to postfix (or use stack-based parsing)
Infix: "3 + 5 * 2" → Postfix: [3, 5, 2, *, +]   
- 3 Build expression tree using stack of nodes  → 
- 4 Perform recursive post-order traversal for evaluation

- 5 Return the final computed result Result: **13** 

3 + 5 * 2



Understanding of Python Logic

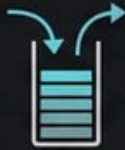
Node Class: Key attributes include 'left', 'right', and 'value' to form a binary structure.



```
class Node:
    def __init__(self, value): ...
```

Stack-Based Construction:

Building the tree by pushing operands and creating parent-child relationships for operators from a stack.



```
class Node:
    def __init__(self, value): self.value = value; self.left = None
```

```
class ExpressionTree:
```

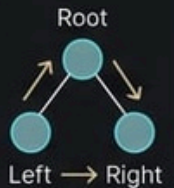
```
    def build_tree(self, expression_tokens): ...
        for tokens in rough.tokens:
            for node in tokens:
                node = createnode((value, right))
                stack.append(node)
                stack.pop()
                stack.push(parents)
            stack.pop()
```

```
    def evaluate(self, node):
        self.evaluate(node, node)
        self.evaluate(traversal(node,), result)
        return ...
```

```
    def postorder_traversal(self, node, result):
        self.postorder_traversal(node, result)
        self.postorder_traversal(node, result)
```

Post-Order Recursive

Evaluation: Children evaluated before parents (Left -> Right -> Root) to calculate expressions efficiently.





THANK YOU

PRANJAL PARIHAR
CM25101